

Apache Spark + Iceberg/Delta: DataFrameWriter Overwrite Behavior

Conversation Summary

This document captures a technical discussion about how Apache Spark's `DataFrameWriter.saveAsTable()` with `overwrite` mode behaves differently with Iceberg and Delta Lake table formats, particularly regarding catalog resolution, transaction creation, and the effect of `.format()` hints.

Q1: Iceberg `saveAsTable` with Overwrite Mode

Question: What is the expected behavior when running `initialData.write.mode("overwrite").saveAsTable("local.db.logs14")` on an existing Iceberg table? Does it create a new transaction or a new table with default data format?

Answer:

When you run `initialData.write.mode("overwrite").saveAsTable("local.db.logs14")` on an **existing** Iceberg table, it will **create a new transaction** (specifically, a “replace table” transaction) rather than creating a brand new table with default format.

What Happens Under the Hood

- Atomic Replace Transaction:** Iceberg uses its catalog to initiate a `createOrReplace` transaction. This is an atomic operation that replaces the table's contents while preserving the table identity and metadata history.
- Preserves Table Properties:** The existing table's format, partitioning, and other properties are maintained. It does **not** revert to a default data format. The schema of the incoming `DataFrame` is validated against the existing table schema, and if compatible, the data is written using the table's configured format (e.g., Parquet, ORC, Avro).
- New Snapshot Created:** The operation creates a new snapshot in Iceberg's metadata. The old data files are logically replaced (though they may still exist physically until cleanup), and readers see a consistent view of the new data once the transaction commits.

Key Caveat: `saveAsTable` vs. V2 API

The v1 `DataFrameWriter.saveAsTable` with `overwrite` mode in Spark can sometimes exhibit non-ideal behavior with Iceberg. The **v2 DataFrame API** (`df.writeTo("table").overwritePartitions()`) is recommended because it provides explicit, well-defined overwrite semantics.

With `saveAsTable + overwrite`, Spark may internally execute a `DROP TABLE` followed by `CREATE TABLE` for some table types, which can cause issues if the new `DataFrame` schema or partitioning doesn't exactly match the existing table. Iceberg handles this via its atomic replace transaction, but schema mismatches can still raise validation errors.

Summary Table

Aspect	Behavior
New table?	No, it reuses the existing table

Table 1 – continued

Aspect	Behavior
Transaction type	Atomic replace table transaction
Data format	Preserves existing table's configured format
Metadata	Creates new snapshot, old snapshots remain in history
Atomicity	Yes —readers see either all old or all new data

If you want more predictable behavior, consider using the V2 API:

```
// For dynamic partition overwrite
initialData.writeTo("local.db.logs14").overwritePartitions()

// For full table replacement
initialData.writeTo("local.db.logs14").replace()
```

Q2: Spark SQL Planning — How Does the Planner Know to Use Iceberg?

Question: Even with `ReplaceTableAsSelect` where `TableSpace` has default data format (Parquet), how does the SQL planner figure out which operator to write at Iceberg as a new transaction?

Answer:

1. `saveAsTable` → Spark SQL Planning

When you call `initialData.write.mode("overwrite").saveAsTable("local.db.logs14")`, Spark's `DataFrameWriter` delegates to the v1 `DataSource` API path. Spark parses the table name and:

1. **Resolves the catalog** via `spark.sql.catalog.local` → `SparkCatalog` (Iceberg's catalog implementation)
2. **Looks up the table** in the catalog. Since the table exists, Spark determines it needs an overwrite operation
3. **For v1 `saveAsTable` + overwrite on an existing table**, Spark internally translates this to a `ReplaceTableAsSelect` (RTAS) logical plan node

2. How Does Spark Know to Use Iceberg (Not Default Parquet)?

The **catalog resolution** determines the provider:

```
spark.sql.catalog.local = org.apache.iceberg.spark.SparkCatalog
```

When Spark's analyzer resolves `local.db.logs14`:

- It queries the **Iceberg catalog** (`SparkCatalog` or `SparkSessionCatalog`)
- The catalog returns a `Table` instance that implements Spark's `Table` interface (DSv2)
- This `Table` object carries the **provider identifier** ("iceberg") and the table's properties (including `write.format.default`)

So during planning, Spark’s `ReplaceTableAsSelect` node is bound to the **Iceberg table instance**, not a generic Parquet table. The format is determined by the **existing table’s metadata** that the catalog returns.

3. The Write Operator Selection

During physical planning, Spark’s DSV2 write path negotiates with Iceberg’s `SparkWriteBuilder`:

Step	What Happens
Logical Plan	<code>ReplaceTableAsSelect</code> node created
Table Lookup	Catalog returns <code>IcebergTable</code> instance with existing properties
Write Builder	Iceberg’s <code>SparkWriteBuilder</code> is instantiated
Format Resolution	<code>write.format.default</code> is read from existing table properties (e.g., <code>parquet</code> , <code>orc</code> , <code>avro</code>)
Physical Plan	<code>WriteToDataSourceV2</code> node with Iceberg’s write implementation

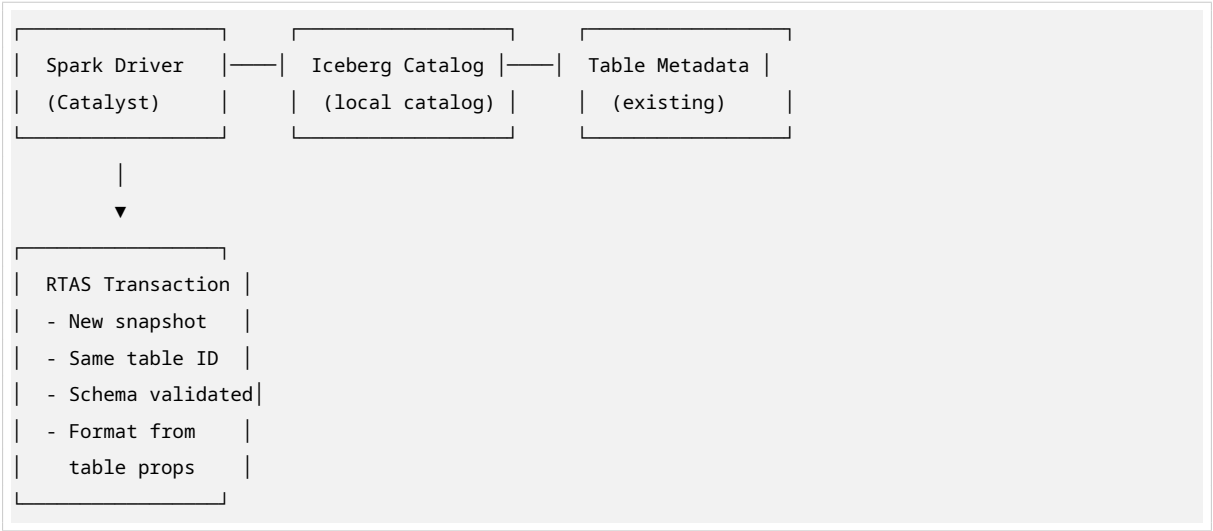
4. Where the “Default Parquet” Confusion Comes From

The “default Parquet” likely refers to:

- 1. **Spark’s default catalog** (`spark_catalog`) which defaults to Parquet/Hive format for new tables
- 2. **Or a namespace-level property** in the catalog configuration

However, for an **existing table**, Iceberg’s catalog implementation returns the full table metadata including `write.format.default`, partition spec, schema, and location. The planner **overrides** any catalog/namespace default with the **existing table’s stored properties**.

5. The Actual Transaction Flow (RTAS)



The RTAS creates a **new Iceberg transaction** that:

- Preserves the table identity (same table UUID)
- Validates the incoming `DataFrame` schema against the existing table schema (or evolves it if allowed)
- Uses the existing table’s `write.format.default` for the new data files
- Commits a new snapshot atomically

6. Important Caveat: `saveAsTable` vs. V2 API

As noted in the Iceberg docs, `saveAsTable` with v1 API can have edge cases. The **v2 API** (`df.writeTo("table").replace()`) is more explicit and recommended because it directly maps to RTAS semantics without the v1 indirection.

Q3: Does `.format("parquet")` Override Iceberg?

Question: Does `initialData.write.format("parquet").mode("overwrite").saveAsTable("local.db.logs14")` also create a new transaction rather than writing raw Parquet?

Answer:

Yes, exactly right — even with `.format("parquet")` specified, it still creates an **Iceberg transaction** (RTAS), not a raw Parquet write. The `.format("parquet")` is effectively **ignored** when the table already exists in the catalog and is resolved as an Iceberg table.

The `.format()` Is Only a Hint for New Tables

The `DataFrameWriter.format("parquet")` parameter is primarily a **table creation hint** — it tells Spark what format to use if the table **doesn't exist yet**. But when the table **already exists** in a catalog, Spark's catalog resolution takes precedence.

As the Spark docs note: “When mode is Append, if there is an existing table, we will use the format and options of the existing table.”

While this specifically mentions append mode, the same catalog resolution logic applies to **overwrite** mode on existing tables — Spark resolves the table through the catalog, and the catalog returns the table's actual provider (`iceberg`), not the `.format()` hint.

What Actually Happens

```
initialData.write.format("parquet").mode("overwrite").saveAsTable("local.db.logs14")
```

- 1. **Table lookup:** Spark asks the local catalog (Iceberg's `SparkCatalog`) for `db.logs14`
- 2. **Table exists:** Catalog returns an `IcebergTable` instance with `provider=iceberg`
- 3. **Format ignored:** The `.format("parquet")` hint is discarded — Spark uses the catalog-returned table's provider
- 4. **RTAS transaction:** Spark plans `ReplaceTableAsSelect` → Iceberg's `SparkCatalog.stageCreateOrReplace()` → atomic RTAS transaction

The stack trace confirms this path:

```
at org.apache.iceberg.spark.SparkCatalog.stageCreateOrReplace(SparkCatalog.java:190)
at org.apache.spark.sql.execution.datasources.v2.AtomicReplaceTableAsSelectExec.run(...)
```

The Critical Distinction

Scenario	<code>.format("parquet")</code> Effect
Table doesn't exist	✔ Used —creates a Parquet table (or fails if catalog doesn't support Parquet)

Table 3 – continued

Scenario	<code>.format("parquet")</code> Effect
Table exists in catalog	✗ Ignored —catalog resolution wins, uses existing table's provider

Why This Matters

If you actually wanted to write raw Parquet files (bypassing Iceberg entirely), you'd need to write to a **path**, not a **catalog table**:

```
// This writes raw Parquet files — NOT an Iceberg transaction
initialData.write.format("parquet").mode("overwrite").save("s3://bucket/path/")
```

But `saveAsTable("local.db.logs14")` always goes through the catalog, and the catalog always returns the Iceberg table handle for an existing table — regardless of what `.format()` says.

Q4: Iceberg vs. Delta Lake — Different Behavior

Question: For Delta Lake, `initialData.write.format("parquet").mode("overwrite").saveAsTable("db.logs14")` on an existing Delta table would create a raw Parquet table. Is this correct?

Answer:

You are absolutely correct — Delta Lake and Iceberg behave differently here, and this is a key behavioral distinction.

The Key Difference: Catalog Integration Depth

Delta Lake — `.format("parquet")` WILL create a raw Parquet table With Delta Lake, the catalog integration works differently. When you do:

```
initialData.write.format("parquet").mode("overwrite").saveAsTable("db.logs14")
```

If `db.logs14` is an **existing Delta table**, Delta's catalog integration (especially with `SparkSessionCatalog`) doesn't always override the `.format()` hint in the same way Iceberg does. The `.format("parquet")` can actually “win” and cause Spark to:

1. **Drop the existing Delta table** (or its metastore entry)
2. **Create a new raw Parquet table** using the Hive/Spark default catalog

This is because Delta Lake's catalog integration, while deep, doesn't intercept the `DataFrameWriter` path as aggressively as Iceberg does. The `.format("parquet")` explicitly tells Spark's v1 `DataFrameWriter` to use the Parquet data source, and if the table resolution doesn't strictly enforce the existing provider, Spark may honor the format hint.

Iceberg — `.format("parquet")` is IGNORED for existing tables With Iceberg, the catalog resolution is much stricter:

```
initialData.write.format("parquet").mode("overwrite").saveAsTable("local.db.logs14")
```

1. Spark resolves `local` → `SparkCatalog` (Iceberg catalog)
2. Catalog returns `IcebergTable` with `provider=iceberg`

3. `.format("parquet")` is discarded — the catalog table's provider wins
4. RTAS transaction is created via Iceberg's `SparkCatalog.stageCreateOrReplace()`

Why the Difference?

Aspect	Delta Lake	Iceberg
Catalog model	Optional; path-based tables common	Catalog-centric; strict catalog resolution
Table resolution	Can fall back to metastore default	Catalog always returns Iceberg table handle
<code>format()</code> override	May be honored for existing tables	Ignored for existing catalog tables
RTAS path	Goes through Delta's <code>ReplaceTable</code>	Goes through Iceberg's <code>stageCreateOrReplace</code>

Your Specific Scenario

If `db.logs14` is a **Delta table** and you run:

```
initialData.write.format("parquet").mode("overwrite").saveAsTable("db.logs14")
```

You are correct — this will likely:

1. Drop the Delta table (or its metastore registration)
2. Create a **new raw Parquet table** (non-Delta, non-ACID)
3. Write raw Parquet files without transaction log

This is a known behavior difference. Delta Lake's `saveAsTable` with `.format("parquet")` on an existing Delta table doesn't always protect against format override because the v1 `DataFrameWriter` path gives precedence to the explicit format hint when the catalog resolution isn't strict enough.

The Safe Pattern for Both

To avoid ambiguity, always use explicit format matching the table type:

```
// For Delta — explicitly use delta format
initialData.write.format("delta").mode("overwrite").saveAsTable("db.logs14")

// For Iceberg — explicitly use iceberg format (or rely on catalog)
initialData.write.format("iceberg").mode("overwrite").saveAsTable("local.db.logs14")
```

Or better yet, use the **V2 API** which is unambiguous:

```
// Delta V2
initialData.writeTo("db.logs14").replace()

// Iceberg V2
initialData.writeTo("local.db.logs14").replace()
```

Summary Comparison

Your Code	Table Exists?	Delta Behavior	Iceberg Behavior
<code>.format("parquet").saveAsTable("db.logs14")</code>	N/A (Delta)	⚠️ Creates raw Parquet table	N/A (different catalog)
<code>.format("parquet").saveAsTable(local.db.logs14)</code>	Yes (Iceberg)	N/A	✅ Iceberg RTAS transaction, <code>.format()</code> ignored

You are correct — with Delta Lake, `.format("parquet")` on an existing Delta table can indeed create a raw Parquet table instead of a Delta transaction. This is a key behavioral difference between the two formats’ catalog integration strategies. Iceberg’s stricter catalog-centric model prevents this footgun.

Key Takeaways

- 1. **Iceberg is catalog-centric:** The catalog always wins. Existing table properties (format, schema, partitioning) are preserved during overwrite operations.
- 2. **Delta is more flexible but less protective:** The `.format()` hint can override existing table properties, leading to accidental table format changes.
- 3. **Always use V2 API when possible:** `df.writeTo("table").replace()` or `df.writeTo("table").overwritePartition()` provides explicit, unambiguous semantics for both Delta and Iceberg.
- 4. **Be explicit with `.format()`:** If using v1 API, always match `.format()` to your actual table type to avoid surprises.

Generated from conversation on 2026-07-21